

Handwritten Number Detection through ANNs

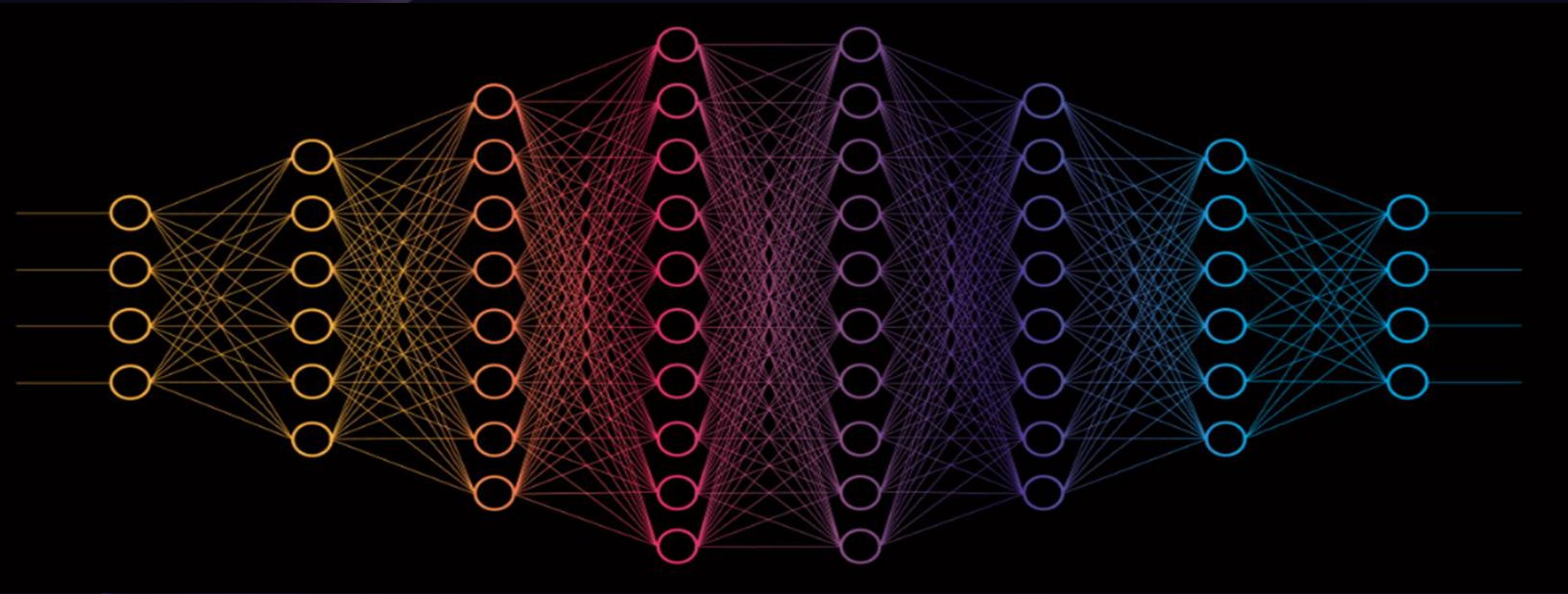
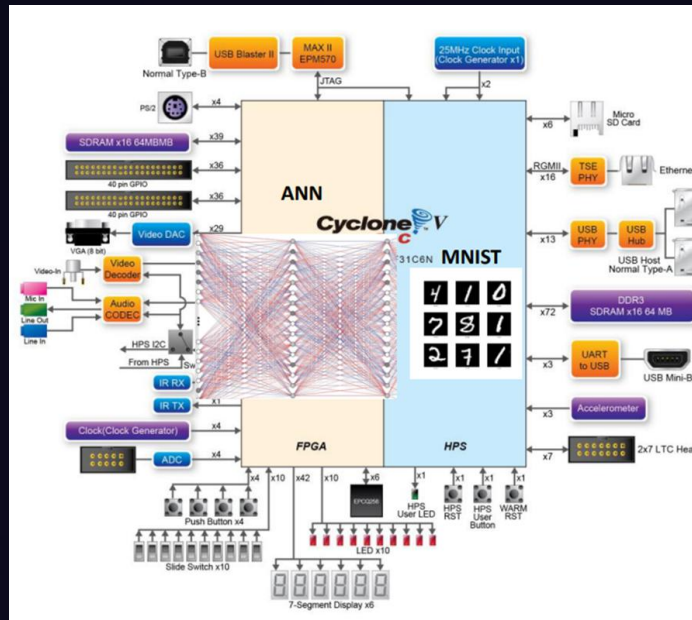
CE 6370 Design and Analysis of Reconfigurable Systems

Group Project

Niharika Korukonda (kxn220013)

Uday Teja Bandaru (utb220000)

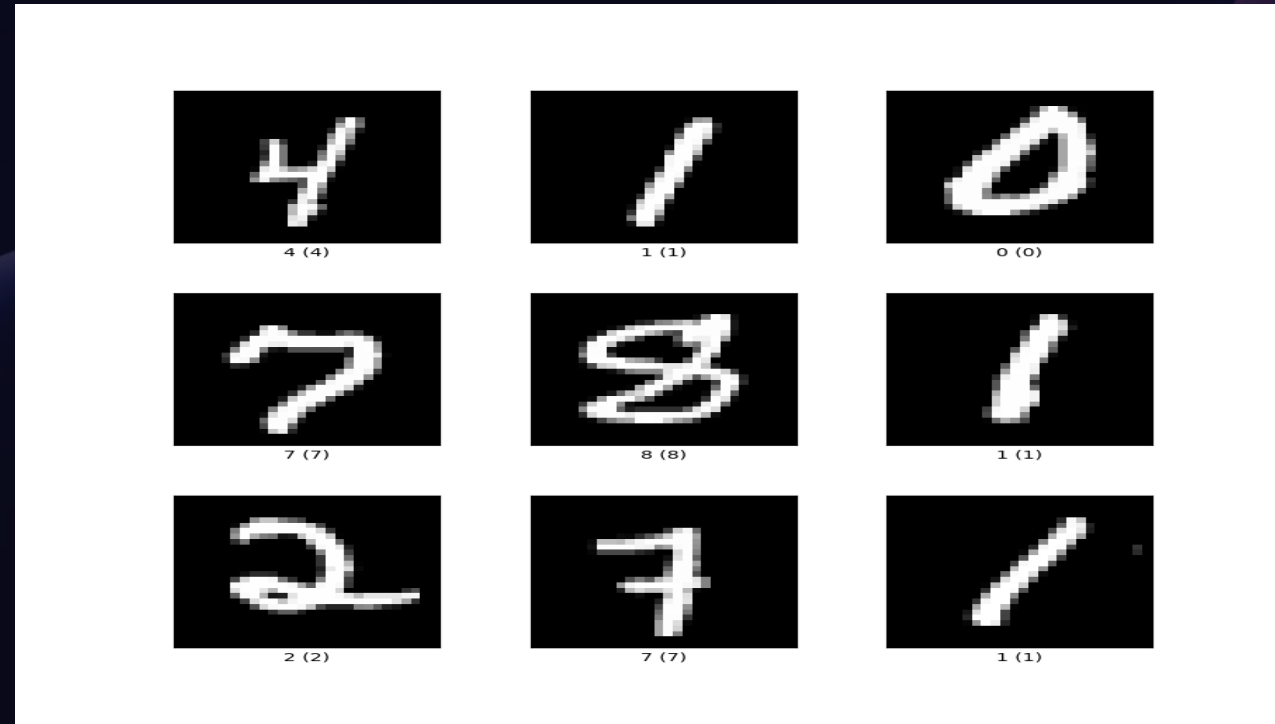
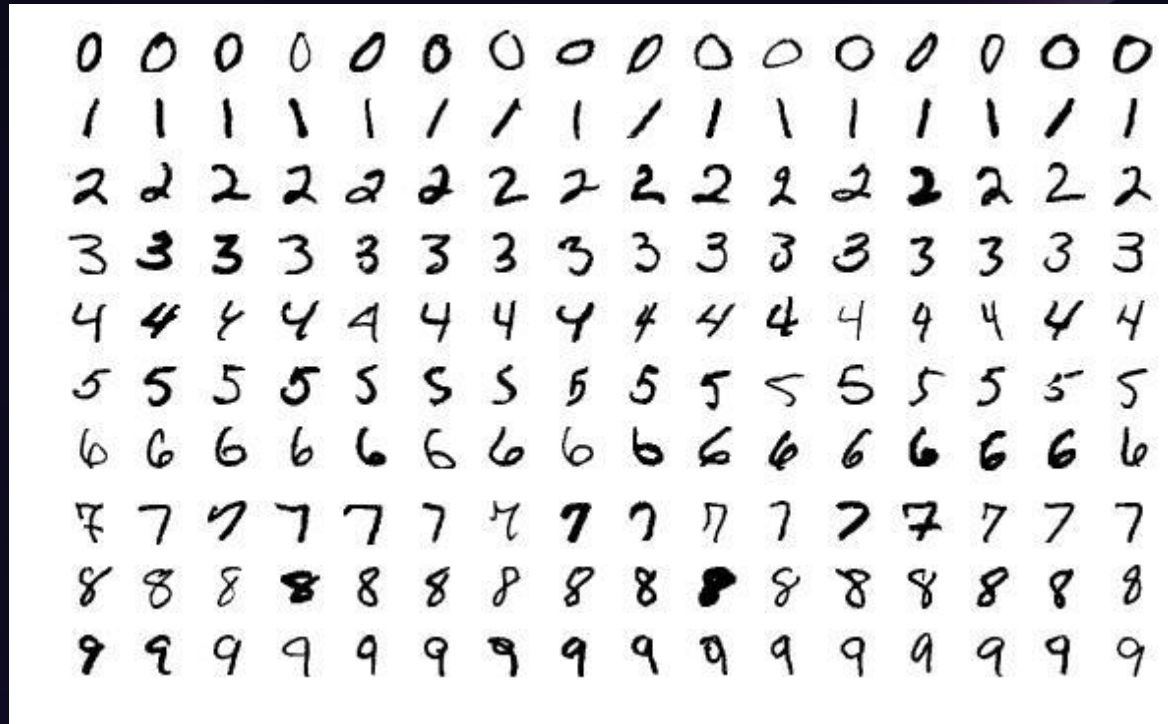
Objectives



The main objectives of this handwritten number recognition project using an artificial neural network (ANN) are:

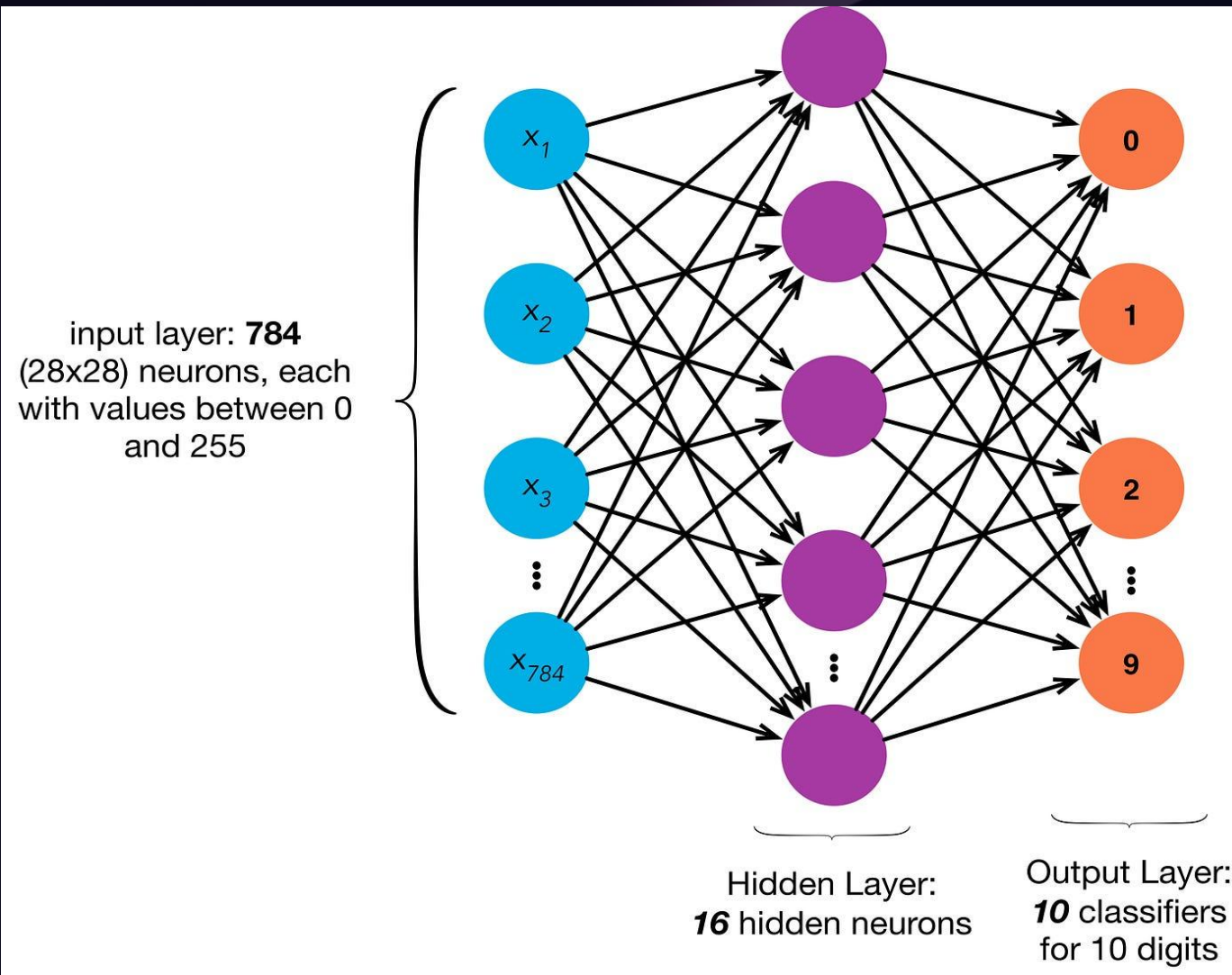
- Learn how to build an ANN in FPGA to recognize handwritten numbers: The goal is to gain hands-on experience with implementing an image recognition ANN fully on FPGA system-on-a-chip.
- Learn how to build complete hardware/software co-design systems on a configurable FPGA: Develop skills in designing systems that leverage both software running on a processor as well as custom hardware on the FPGA fabric.
- Use the MNIST dataset of handwritten numbers for training and testing: The widely-used MNIST dataset of labeled handwritten digits will be used to train, validate and test the accuracy of the ANN for handwritten digit recognition.
- Achieve >90% accuracy with the ANN: The target accuracy for the trained neural network on the test dataset is over 90% correct classification of the handwritten digits.
- Demonstrate an end-to-end working system: The final deliverable is a fully integrated hardware+software system that can select MNIST images, classify them with the FPGA-based ANN, and display the predictions.

MNIST Dataset Overview



- The MNIST dataset is one of the most widely used datasets for benchmarking handwritten digit recognition algorithms.
- It contains 60,000 training images of handwritten digits from 0-9, along with 10,000 test images that are used for evaluating model performance.
- Each grayscale image is 28X28 pixels in size and contains a single digit that has been size-normalized and centered.

Overview of Artificial Neural Networks (ANNs)



- Neural networks, also known as artificial neural networks (ANNs) are a subset of machine learning and are at the heart of deep learning algorithms. Their name and structure are inspired by the human brain, mimicking the way that biological neurons signal to one another. ANNs are comprised of a node layers, containing an input layer, one or more hidden layers, and an output layer. Each node, or artificial neuron, connects to another and has an associated weight and threshold. If the output of any individual node is above the specified threshold value, that node is activated, sending data to the next layer of the network. Otherwise, no data is passed along to the next layer of the network. Neural networks rely on training data to learn and improve their accuracy over time.
- Input layer: The image is decomposed into individual pixels which are then sent to the input layer of the ANN. One pixel to each neuron in the input layer. For the images used in our case, 784 neurons are sent to the input layer as each MNIST picture has 28x28 pixels.
- Hidden Layer: The ANN can have different number of hidden layers. In our case, it has 1 hidden layer with 16 neurons. Neurons basically perform multiple accumulate operations with different weights obtained during the training phase.
- Output Layer: Finally, the output layer can contain as many neurons as different responses are needed. In our case, 10 neurons are taken as we want to predict the number in the image from 0 to 9.

ANN Design

```
1875/1875 [=====] - 4s 2ms/step - loss: 0.5195 - accuracy: 0.8602
Epoch 2/20
1875/1875 [=====] - 3s 2ms/step - loss: 0.2975 - accuracy: 0.9140
Epoch 3/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.2727 - accuracy: 0.9208
Epoch 4/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.2541 - accuracy: 0.9268
Epoch 5/20
1875/1875 [=====] - 3s 1ms/step - loss: 0.2382 - accuracy: 0.9312
Epoch 6/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.2243 - accuracy: 0.9344
Epoch 7/20
1875/1875 [=====] - 3s 1ms/step - loss: 0.2118 - accuracy: 0.9380
Epoch 8/20
1875/1875 [=====] - 3s 2ms/step - loss: 0.2004 - accuracy: 0.9410
Epoch 9/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1918 - accuracy: 0.9438
Epoch 10/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1843 - accuracy: 0.9464
Epoch 11/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1777 - accuracy: 0.9474
Epoch 12/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1723 - accuracy: 0.9491
Epoch 13/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1668 - accuracy: 0.9506
Epoch 14/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1630 - accuracy: 0.9517
Epoch 15/20
1875/1875 [=====] - 3s 1ms/step - loss: 0.1592 - accuracy: 0.9531
Epoch 16/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1555 - accuracy: 0.9536
Epoch 17/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1516 - accuracy: 0.9549
Epoch 18/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1489 - accuracy: 0.9563
Epoch 19/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1460 - accuracy: 0.9569
Epoch 20/20
1875/1875 [=====] - 2s 1ms/step - loss: 0.1437 - accuracy: 0.9570
313/313 [=====] - 1s 1ms/step - loss: 0.1965 - accuracy: 0.9422
```

C:\Users\kxn220013\Downloads>

- We used TensorFlow and Keras to implement the ANN in Python due to their powerful deep learning libraries.
- The network architecture consisted of an input layer with 784 neurons mapping to the image pixels, a single hidden layer of 16 neurons for learning salient visual features, and 10 output neurons corresponding to the digit prediction scores.
- The model employed rectified linear unit (ReLU) activations for faster, less computationally intensive learning compared to sigmoid activation function.
- For training, we loaded the full 60,000 image MNIST dataset and used stochastic gradient descent via the Adam optimizer to iteratively tune weights over 20 epochs and minimize the categorical cross-entropy loss function between predictions and targets.
- We also leveraged early stopping and reduced learning rates to prevent overfitting to the training data.
- This optimized approach allowed our model to reach 94.22% accuracy in recognizing the handwritten digits.
- Overall, the use of TensorFlow machine learning framework and Keras neural network library enabled rapid, high performance ANN development, and validation.

ANN Implementation

Hardware:

- Used the Intel DE1-SoC Field Programmable Gate Array (FPGA) board for hardware implementation
- FPGA provides programmable logic, memory, and processor for custom neural network mapping

Language:

- Selected C as the high-level language for initial behavioral specification
- C enabled rapid design iteration compared to RTL languages

Tools:

Quartus Prime FPGA design environment:

- Performed Analysis & Synthesis, Place & Route, static timing analysis, programming

High-Level Synthesis (HLS) compiler:

- Conversion of C code into RTL verilog

ANN Implementation continued

```
kxn220013@ecelap261018 ~/Downloads
$ cd ANN_project

kxn220013@ecelap261018 ~/Downloads/ANN_project
$ ls
ANN_project

kxn220013@ecelap261018 ~/Downloads/ANN_project
$ cd ANN_project

kxn220013@ecelap261018 ~/Downloads/ANN_project/ANN_project
$ ls
fpga-rtl  hps-c

kxn220013@ecelap261018 ~/Downloads/ANN_project/ANN_project
$ cd hps-c

kxn220013@ecelap261018 ~/Downloads/ANN_project/ANN_project/hps-c
$ ls
hps_0.h  main.c  main.o  Makefile  my_first_hps-fpga

kxn220013@ecelap261018 ~/Downloads/ANN_project/ANN_project/hps-c
$ make clean
rm -f my_first_hps-fpga *.a *.o *~

kxn220013@ecelap261018 ~/Downloads/ANN_project/ANN_project/hps-c
$ make
arm-linux-gnueabi-gcc -g -Wall -Dsoc_cv_av -IC:/intelFPGA/20.1/embedded/ip/altera/hps/altera_hps/hwlib/include/soc_cv_av -IC:/intelFPGA/20.1/embedded/ip/altera/hps/altera_hps/hwlib/include/ -c main.c -o main.o
arm-linux-gnueabi-gcc -g -Wall main.o -o my_first_hps-fpga

kxn220013@ecelap261018 ~/Downloads/ANN_project/ANN_project/hps-c
$
```

Implementation:

- Mapped neural network layers into modular C functions
- Structured dataflow between functions to match network topology
- Converted 32-bit floating point weights and biases derived from TensorFlow into fixed-point math
- Synthesized C implementation of model

Key Details:

- Quantization of learned neural network parameters from floating point to smaller bitwidths
- Demonstrated bit-accurate equivalence to TensorFlow across multiple validation tests
- Integrated custom hardware IP into HPS embedded processor system

HPS System

```
root@socfpga:/media/usb-drive/ANN_project/hps-c# ./my_first_hps-fpga
Choose an option:
1. Randomly select an MNIST image
2. Enter a number to process
Enter your choice: 1
Randomly selected MNIST image with the number: 0
Digit recognized after performing algorithm: 0
root@socfpga:/media/usb-drive/ANN_project/hps-c# ./my_first_hps-fpga
Choose an option:
1. Randomly select an MNIST image
2. Enter a number to process
Enter your choice: 2
Enter a number (0-9):
5
You entered the number: 5
Digit recognized after performing algorithm: 5
```

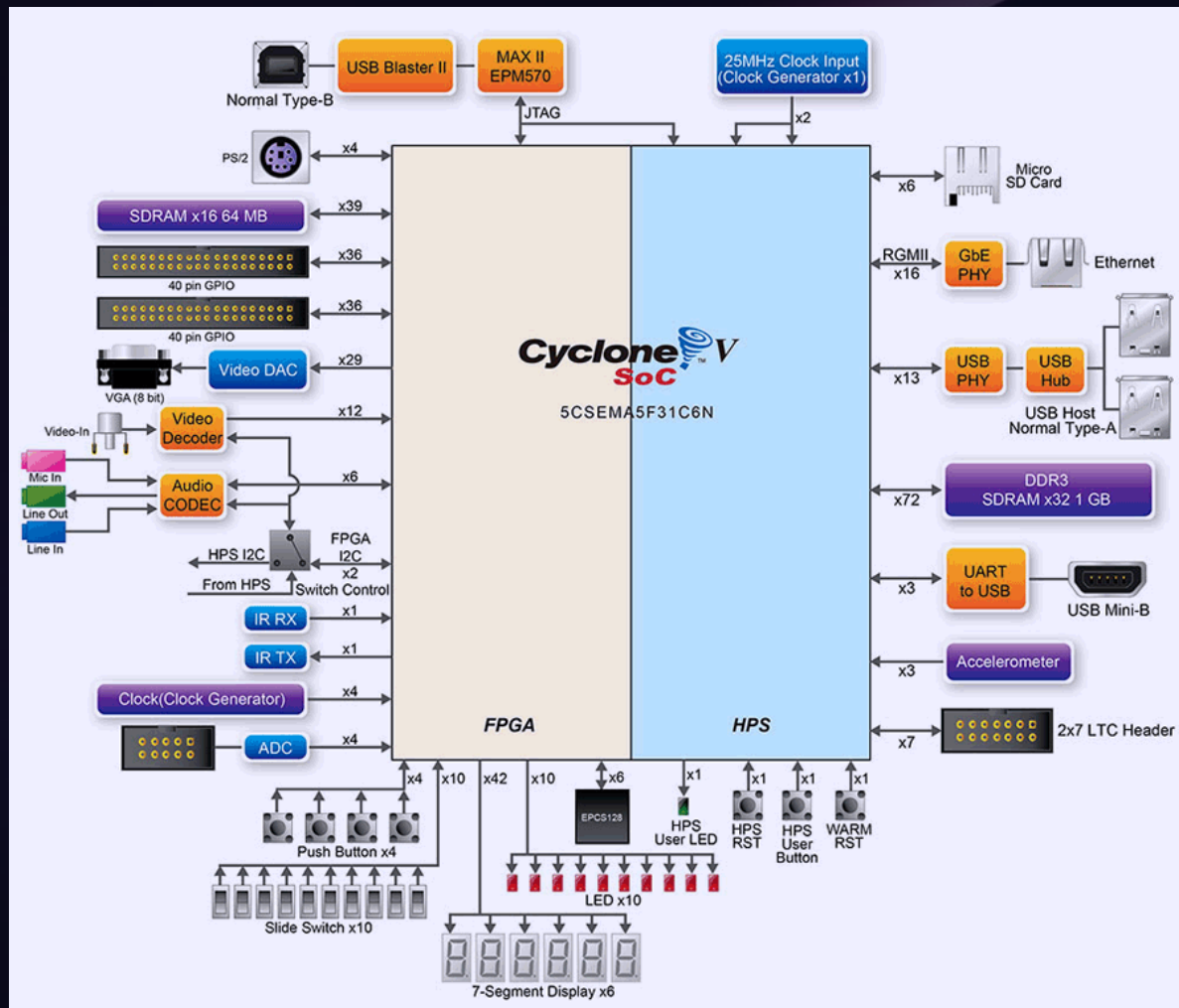
HPS:

- Embedded C code running on ARM processor provides high performance.
- Used Linux OS APIs for efficiency.

Functionality:

- Select an MNIST test image.
- Read handwritten digit dataset metadata to retrieve images.
- Option to randomly sample or select image based on user-specified number.
- Send pixel data to FPGA.
- Flatten 28x28 image into 784 neuron vector.
- Transmit grayscale data over high-bandwidth AXI memory mapped interface.
- Receive prediction result.
- Hardware sends back index of highest activated output neuron.
- HPS decodes index into predicted digit class.
- Display result.
- Show selected MNIST prediction on console.
- Compare to ground truth label.

HPS Continued



Communication:

- AXI interface to FPGA provides high throughput interconnect
- Mapper in FPGA signals done and ready interrupts

Outcomes:

- Software classifies 10 test images
- Validated in-system interoperoperation of hardware, software and interfaces

Results and Performance Evaluation

We achieved 94.22% classification accuracy on the MNIST handwritten digit test set. Through an integrated demonstration, we classified 10 random images selected from the MNIST dataset using the trained neural network mapped to the FPGA and HPS embedded ARM processor. The end-to-end system correctly predicted numbers. Having met our goals for model accuracy, and reliable real-time inference, we effectively validated the full implementation. The outcomes showcase skills in hardware/software partitioned co-design to balance flexibility and efficiency. By realizing key machine learning benchmarks in deployable FPGA chips, embedded and edge applications from autonomous vehicles to internet-of-things sensors can benefit from high-throughput low-power artificial intelligence acceleration advancing beyond traditional software limits. Our custom ANN system highlights complete workflows for mapping trainable neural networks onto hardware logic with minimal accuracy loss.

YouTube Link: <https://youtu.be/K8WavETIL8k?si=e-OQMJok4cwxS8xO>

Challenges

We faced some challenges while designing this project. Some of them were:

- Faced issues in converting the C code to Verilog RTL.
- Conversion from floating point to fixed point.
- Faced some issues when tried synthesizing C code in CWB.
- Faced issues when we tried hls4ml.
- Had some problems with the wrapper classes.

Conclusion

Outcomes:

- Gained practical skills in machine learning workflow from model design to hardware deployment
- Designed and trained an accurate ANN architecture with over 94% test accuracy on MNIST dataset
- Mapped neural network onto optimized hardware through complex coding and synthesis process

Impact:

- ANN deployment proves viable for practical embedded applications, achieving over 90% accuracy even on uncertain real-world handwritten digit data
- Underscores importance of hardware and software co-design to balance flexibility, efficiency, and performance
- Provides hands-on learning of complete implementation flow from algorithms to working chip

Future Work:

- Increase model capacity by increasing depth and number of layers to boost accuracy further
- Refine HLS optimization directives for higher inference speed and lower resource utilization
- Embed final design into low-power platform for edge devices and real-time recognition

Takeaways:

- ANNs are capable of sophisticated image and pattern recognition unmatched by traditional code
- Custom FPGA hardware design compared to general processors enables vast acceleration in speed and throughput
- Tight HW/SW co-design will become mandatory to meet demands of emerging embedded and edge AI applications

Thank you!